

FELLOWS' ENGINEERING GUIDE

AI-Native Engineering Sprint — CampusFlow

6-HOUR TEAM CHALLENGE | 2 FELLOWS PER TEAM | 100 MARKS | PYTHON + AI LEARNING + GITHUB

YOUR MISSION: Build a working product. Learn with AI. Verify everything. Review each other's code before merging.

This is your practical, step-by-step guide. Read the mission once, then follow the six-hour checkpoints. You are not expected to know every concept in advance; you ARE expected to learn deliberately, test your decisions, and collaborate professionally.

START HERE | WHAT SUCCESS LOOKS LIKE

- Your team delivers a command-line helpdesk app called CampusFlow with ticket creation, priority calculation, assignment, status changes, queue, reporting, JSON storage, and automated tests.
- Both fellows independently contribute Python code and tests on their own Git branches.
- Each fellow authors a pull request (PR), reviews the other fellow's PR, and receives approval before merging into main.
- Each fellow records at least three verified AI learning interactions, including one AI suggestion independently corrected, improved, or rejected.
- Your team runs the complete test suite on main and demonstrates the project together within six hours.

NON-NEGOTIABLE: A project that works but has no genuine GitHub review, no tests, or no evidence of AI-assisted learning is NOT a complete submission.

1 | UNDERSTAND THE PRODUCT YOU ARE BUILDING

1.1 The real-world problem

Learn2Earn campus staff receive technical support requests: Wi-Fi outages, faulty laptops, broken development environments, and inaccessible platforms. Informal reports get lost. Build CampusFlow, a Python command-line ticket manager that records problems, prioritizes work, assigns responsibility, and tracks progress.

Think like engineers: Which inputs does the system need? What must it calculate? What could go wrong? What should be preserved after the program closes?

1.2 Ticket fields (required)

Each ticket stores: id, title, category, urgency, affected_users, priority, status, and assigned_to.

EXAMPLE (Python dictionary):

```
{
  "id": "T001",
  "title": "Campus Wi-Fi is down",
  "category": "Network",
  "urgency": "high",
  "affected_users": 15,
  "priority": "critical",
  "status": "open",
  "assigned_to": None
}
```

- Ticket IDs are unique and automatically generated (T001, T002, T003, ...).
- Allowed categories: Network, Hardware, Software, Other.
- Allowed urgency levels: low, medium, high.
- Title must not be blank. affected_users must be a positive integer (not zero, a negative number, a decimal, or text).
- Accept and normalize valid case variations consistently; display useful errors for invalid input.

1.3 Calculate priority exactly as specified

Your function must check the following rules IN THIS ORDER and apply the first match:

RULE 1 high urgency AND 10 or more affected users → critical

RULE 2 high urgency OR 10 or more affected users → high

RULE 3 medium urgency OR 3 or more affected users → medium

RULE 4 all remaining valid tickets → low

Important: This engine is ordinary Python business logic, not an AI model. You use AI to LEARN and BUILD, not as the priority engine itself.

1.4 Build these seven features

- F1 — CREATE: Collect ticket details, validate inputs, calculate priority, assign a unique ID and store the record.
- F2 — LIST / VIEW: Show all tickets and details for one chosen ticket.
- F3 — ASSIGN: Assign a ticket to a non-empty staff member name; reject unknown ticket IDs.
- F4 — WORKFLOW: Support open → in_progress → resolved. Do not move an unassigned ticket into in_progress. A resolved ticket may only be modified after an explicit reopen that sets it to open.
- F5 — WORK QUEUE: Show open, unresolved tickets in critical → high → medium → low order; break priority ties using the earlier numeric ticket ID.
- F6 — REPORTS: Show total tickets and breakdowns by status and priority; handle zero tickets correctly.
- F7 — PERSISTENCE: Save to JSON and reload on startup. Missing file should allow a fresh start; malformed JSON should produce a clear error rather than silently erase data. IDs must remain unique after reload.

Create a menu that keeps running until Exit. The interface can be text-only. Do not spend sprint time building a GUI or website.

1.5 Decide what 'done' means

- ☐ All seven features are available from a functioning CLI menu.
- ☐ The specified priority examples produce correct results.
- ☐ Invalid inputs and transitions are rejected WITHOUT corrupting stored tickets.
- ☐ Tickets survive a restart using JSON.
- ☐ At least eight meaningful automated tests pass on the integrated main branch.
- ☐ Both PRs have genuine partner review, approval, and merge evidence.

2 | BEFORE CODING: WORK AS A TWO-PERSON ENGINEERING TEAM

2.1 Agree on responsibilities

Suggested ownership (you may agree on equivalent balanced ownership):

ENGINEER A: ticket creation, input validation, and priority engine; write matching automated tests.

ENGINEER B: assignment, lifecycle, queue, and reports; write matching automated tests.

BOTH: agree on data structures and function contracts; integrate JSON persistence; review each other's code; complete AI learning logs and technical defence.

Each person must write real application code AND tests. One person doing all implementation while the other only watches or writes the README does not satisfy the team rules.

2.2 Agree on function contracts

Before writing features, discuss these questions together:

- ☐ Where is the ticket collection stored and how is it passed between functions?
- ☐ How does a function signal invalid input (exception or structured error), and how does the CLI display it?
- ☐ What is the expected return value when a ticket is created, assigned, or updated?
- ☐ Which module owns loading/saving JSON and how do other modules call it?
- ☐ How can we test business logic without asking for interactive input inside each test?

Record your agreements in docs/design-decisions.md. Keep decisions simple; the goal is agreement, not perfect architecture.

2.3 Example project layout (suggested)

```
campusflow/  
├─ README.md  
├─ .gitignore  
├─ main.py  
├─ campusflow/  
│   └─ __init__.py  
│   └─ tickets.py
```

```

|   |─ workflow.py
|   |─ storage.py
|   └─ reports.py
|─ tests/
|   |─ test_tickets.py
|   |─ test_workflow.py
|   └─ test_storage.py
└─ docs/
    |─ ai-learning-log.md
    └─ design-decisions.md

```

Use Python's standard library, including unittest and json. You do not need a framework, database, AI SDK, or API key. Exclude runtime files like data/tickets.json and developer-local files from Git.

3 | SIX-HOUR GUIDED SPRINT

PHASE 1 — 00:00–00:20 | SET UP THE REPOSITORY

Together, create ONE GitHub repository named campusflow-<team-id> (or a similarly clear name). Invite your partner as a collaborator and ensure both can push their own branches. Initialize main with a README and .gitignore. Clone the repository to each computer.

- ☐ Both fellows can open the repository and see main.
- ☐ Both fellows have Git configured with their own account and identity.
- ☐ README states the project goal and team members.
- ☐ .gitignore includes .venv/, __pycache__/, *.pyc, .env and data/tickets.json.

Evidence checkpoint: GitHub repository URL, both collaborators, initial commit.

PHASE 2 — 00:20–00:50 | PLAN BEFORE IMPLEMENTING

Read the seven features together. Create GitHub Issues with clear acceptance criteria, for example: “Create a ticket with valid data” and “Reject affected_users=0”. Decide which fellow owns each feature and agree on shared function contracts.

- ☐ A GitHub Issue exists for each major work area (combine small related tasks where practical).
- ☐ Each fellow has a clear coding task and a matching test responsibility.
- ☐ docs/design-decisions.md records agreed interfaces and ownership.
- ☐ You can explain how both feature branches will work together before merging.

Evidence checkpoint: Issue list and design-decisions document.

PHASE 3 — 00:50–01:30 | LEARN WITH AI BEFORE YOU BUILD

Individually choose TWO topics you are unsure about (for example JSON persistence, Python exception handling, unittest assertions, safe list operations, Git branches, or PR reviews). Do not ask AI to build the full CampusFlow project.

Complete a small learning loop for each topic:

1. State what you currently understand and the specific point you do not understand.
2. Ask AI for an explanation and a SMALL unrelated example.
3. Run or adapt a tiny experiment yourself; predict the result before executing it.
4. Verify the result from the output, a test, or official reference documentation.
5. Explain what you learned in your own words and how it will help your feature.

This is the first part of your AI learning log. Complete at least one additional entry during coding/debugging to reach three per fellow.

- ☐ Each fellow has at least two initial AI learning entries or experiments.
- ☐ Each fellow can explain a concept learned without repeating an AI response word for word.

Evidence checkpoint: two learning experiments per fellow.

PHASE 4 — 01:30–03:55 | IMPLEMENT, TEST, COMMIT

Work on separate feature branches. Build the agreed functions and write automated tests. Keep changes focused; make meaningful incremental commits. Use AI for explanations, investigating test failures, edge cases and design critique; VERIFY every suggestion.

Suggested cadence: implement a small requirement → test locally → inspect git diff/status → commit → continue.

- ☐ Both fellows have meaningful code commits on their own branches.
- ☐ Core functionality is separated enough to unit-test.
- ☐ At least eight meaningful tests are planned and being implemented.
- ☐ Invalid-input cases are tested, not just successful paths.
- ☐ Each fellow has at least three AI log entries, with one documented critical evaluation.

Evidence checkpoint: published feature branches, commits, test files, AI logs.

PHASE 5 — 03:55–05:00 | OPEN AND REVIEW PULL REQUESTS

Engineer A opens a PR from A's feature branch to main. Engineer B reviews it. Engineer B opens a PR and Engineer A reviews it. Each reviewer must inspect real code, check requirements, run relevant tests where practical, and leave at least TWO substantive review observations before approval.

If there is a real blocking problem, request changes. The author commits a correction to the same branch, pushes it and asks for re-review. Only merge after approval and passing relevant tests.

- ☐ Each fellow authored at least one feature PR.
- ☐ Each fellow reviewed the other fellow's PR.
- ☐ PRs include a clear description, Issue reference and test evidence.
- ☐ Review comments are meaningful (not merely "LGTM" or "good code").
- ☐ Every blocking issue is resolved and the final reviewer approval is visible.
- ☐ No one self-approved or merged an unreviewed PR.

Evidence checkpoint: links to two reviewed PRs. If the second branch is behind main after the first merge, update it from main, resolve conflicts, rerun tests and obtain approval on the final changes.

PHASE 6 — 05:00–05:40 | INTEGRATE AND POLISH

Run the whole project from latest main. Run every automated test, verify JSON round-trip persistence and unique IDs after reload, and check an invalid status transition. Fix integration defects through a new reviewed PR—not by bypassing the workflow.

- ☐ Both approved feature PRs are merged into main.
- ☐ The CLI works from main on a clean run.
- ☐ Automated tests are passing on main.
- ☐ README contains clear run/test instructions and honest known limitations.
- ☐ Learning log and design-decisions documents are committed.

Evidence checkpoint: main commit, passing test output, complete documentation.

PHASE 7 — 05:40–06:00 | DEMONSTRATE AND DEFEND

You have 20 minutes for the team's final demonstration and technical explanation. Both fellows must speak about their own work.

Demonstration route: create several tickets → show priority output → reject an invalid ticket → assign and progress a ticket → reject an invalid transition → display ordered queue/report → save and reload → run automated tests → show both PR reviews.

- ☐ Engineer A explains one implementation decision and a relevant test.
- ☐ Engineer B explains one implementation decision and a relevant test.
- ☐ Both fellows explain one AI suggestion they verified, changed or rejected.
- ☐ Both fellows can show the PR they authored and the PR they reviewed.

Evidence checkpoint: shared repository and PR links, test results, and individual explanations.

4 | PROFESSIONAL GIT WORKFLOW: COPYABLE GUIDE

4.1 Use a proper branch name

Format: <type>/<issue-number>-<short-description>. Lowercase words, hyphens between words, and an actual GitHub Issue number when available.

GOOD BRANCH NAMES

`feat/1-ticket-creation`

`feat/2-status-workflow`

`fix/7-invalid-status-transition`

`test/8-ticket-validation`

`docs/9-readme-setup`

AVOID

`john-work`

`branch2`

```
final-final  
new-stuff
```

4.2 Create a feature branch from latest main

```
git clone <repository-url>  
cd <repository-folder>  
git switch main  
git pull origin main  
git switch -c feat/1-ticket-creation
```

4.3 Inspect and commit a small piece of work

```
git status  
git diff  
git add campusflow/tickets.py tests/test_tickets.py  
git commit -m "feat: implement ticket creation"  
git push -u origin feat/1-ticket-creation
```

Professional commit examples: feat: add ticket creation validation; test: cover priority boundaries; fix: reject invalid status transitions; docs: document running tests. Stage relevant files rather than blindly including unrelated changes.

4.4 Open your PR on GitHub

On GitHub: select your pushed branch → Compare & pull request → base: main / compare: your feature branch → add a concise title → fill out description and test evidence → link related Issue → request review from your partner.

You must not create a PR with no meaningful description, nor merge it before partner review.

4.5 Review your partner's PR professionally

On GitHub: open Pull requests → your partner's PR → Files changed. Read the actual code; compare the implementation with the requirements; inspect test additions. Comment on specific lines where possible. To test it locally, fetch the branch and run its tests in a non-editing checkout:

```
git fetch origin  
git switch --detach origin/feat/2-status-workflow  
python -m unittest discover -s tests -v  
# Return to your usual branch  
git switch main
```

The feature branch shown is an example. Replace it with the actual name of the PR branch. Detached checkout lets you inspect the exact remote snapshot without modifying your partner's branch.

Your review should explain a concern or a verified check. Examples: “If `affected_users` is zero, this function currently accepts it; please add a test and validation.” or “I ran the tests and confirmed the `high+10` boundary gives critical.”

If blocking issues exist, choose Request changes. If the change is ready, choose Approve. After your partner updates the branch, review the final code before approval. The author cannot approve their own PR.

4.6 Update a feature branch when main changes

If your partner's first PR is merged before yours, update your branch and retest. One beginner-friendly approach:

```
git switch main
git pull origin main
git switch feat/2-status-workflow
git merge main
# Resolve any conflicts by reading both changes carefully.
python -m unittest discover -s tests -v
git add <resolved-files>
git commit    # Only if a merge commit is pending.
git push
```

A merge conflict is not a disaster: inspect markers, decide the correct combined behaviour with your partner, remove the markers, rerun tests, and push the resolved branch. If unsure, ask for explanation—not for someone to erase the conflict blindly.

Professional standard: no direct work on main, no unreviewed merge, no secrets in commits, no fabricated tests or reviews.

5 | HOW TO USE AI LIKE AN ENGINEER

5.1 Your learning cycle

Understand the task → identify what you don't know → ask AI a focused question → experiment in a small isolated example → verify → apply the learning → test your application.

5.2 Prompts you may use as starting points

LEARNING: “Explain `json.dump()` and `json.load()` with a five-line example unrelated to CampusFlow. Then quiz me. I want to write my own code.”

DESIGN: “I plan to store tickets in a list of dictionaries. What trade-offs should I consider for updates, lookup, and testing? Do not code my project.”

DEBUGGING: “Here is my failing test and traceback. Help me isolate likely causes. Give experiments first, not a complete rewrite.”

TESTING: “Here are my acceptance criteria. Which edge cases should I test, and why? Give descriptions rather than finished project tests.”

CODE REVIEW: “Here is a small diff. Point out risks and questions for a human reviewer. I will verify your claims myself.”

5.3 Individual AI learning-log entry template

```
## Fellow: <your name>
### Interaction #1 – <concept/topic>
Problem: <what I needed to learn>
My initial understanding: <in my own words>
Prompt to AI: <my actual prompt>
Useful AI guidance: <brief summary>
My independent experiment/test: <what I ran and observed>
Verification source or result: <test result/reference>
Decision: <accepted, improved or rejected and why>
Related file/commit: <path, test name, SHA or PR link>
What I can now explain without AI: <my words>
```

Do this for at least THREE meaningful interactions per fellow. At least ONE must document how you corrected, improved or rejected an AI suggestion after checking it. Keep the log truthful; do not fabricate a flaw or pretend to have verified something you did not run.

AI usage is required, but copy-pasting a full app without understanding or testing it does not count as engineering learning.

6 | AUTOMATED TESTING: LEARN TO PROVE IT WORKS

You must write at least EIGHT meaningful automated tests using unittest. A useful test has a clear expectation and an assertion. Include normal cases, invalid inputs, boundaries, and persistence.

6.1 Required acceptance scenarios (prioritize these)

- ☐ High urgency + 12 affected users → critical priority.
- ☐ High urgency + 2 affected users → high priority.
- ☐ Low urgency + 4 affected users → medium priority.
- ☐ Low urgency + 1 affected user → low priority.
- ☐ Zero affected users → rejected.
- ☐ Unassigned ticket cannot move to in_progress.
- ☐ Assigned ticket can progress through open → in_progress → resolved.
- ☐ Open-ticket queue sorted by priority and then numeric ID.
- ☐ Save and reload JSON preserves existing tickets.
- ☐ Create a ticket after reload without duplicating IDs.

Recommended extra cases: empty report, unknown ticket ID, blank title, invalid urgency, invalid transition, reopening, corrupted JSON, and ties in the work queue.

6.2 Test command

```
python -m unittest discover -s tests -v
```

Do not just say 'the tests pass'. Run them and inspect the output. A failing test is valuable feedback; fix the underlying requirement and run again. Do not modify tests merely to make a broken implementation appear correct.

7 | PR DESCRIPTION TEMPLATE

Copy this outline into each GitHub pull request and complete it honestly.

PR TITLE: feat: <short meaningful change>

Related Issue: Closes #<number>

WHAT CHANGED?

- <features and files modified>

HOW DOES IT WORK?

- <approach and key decisions>

HOW DID I TEST IT?

- Command: `python -m unittest discover -s tests -v`

- Actual result: <pass/fail details>

- Edge cases: <what I verified>

WHAT DID I LEARN WITH AI?

- Concept: <topic>

- My verification: <experiment or test>

- Changed/rejected advice: <if applicable>

REVIEWER NOTES

- What I want my partner to check: <specific risk/question>

KNOWN LIMITATIONS

- <honest known gaps>

8 | README CHECKLIST

- ☐ Project title and clear problem statement.
- ☐ Both fellows' names/roles and a short contribution summary.
- ☐ Python version and setup instructions.

- ☐ How to run the CLI and available menu options.
- ☐ How to run the tests and what they cover.
- ☐ How ticket priority and status rules work.
- ☐ Where tickets are saved and how to start fresh.
- ☐ Known limitations and future improvements.
- ☐ Links to both reviewed PRs (or a GitHub PR index).

9 | ASSESSMENT: HOW YOUR WORK IS SCORED

FUNCTIONAL IMPLEMENTATION — 40 marks

- 15 — Ticket creation and priority engine.
- 10 — Assignment and status management.
- 10 — Open-ticket queue and reporting.
- 5 — JSON persistence.

TESTING AND RELIABILITY — 15 marks

- 15 — Automated tests, meaningful assertions, invalid-input cases, and reliable behaviour.

PROFESSIONAL GIT WORKFLOW — 20 marks

- 5 — Branch naming and commit quality.
- 5 — PR descriptions and traceable individual work.
- 10 — Substantive partner reviews, approval, and correct merges.

AI-NATIVE ENGINEERING — 20 marks

- 5 — Conceptual learning with AI.
- 5 — Independent experiments and verification.
- 5 — Critical evaluation of AI suggestions.
- 5 — Quality and credibility of learning logs.

TECHNICAL DEFENCE — 5 marks

- 5 — Working explanation, demonstration, README and individual understanding.

TOTAL: 100 marks. Feature functionality and integrated test outcomes are assessed as team work. Git contributions, AI learning and individual understanding are assessed per fellow; you do not automatically earn another fellow's process marks.

10 | FINAL SUBMISSION CHECKLIST

- ☐ Paste the GitHub repository link (main branch).
- ☐ Paste Engineer A's PR link and Engineer B's PR link.
- ☐ Confirm both PRs have real cross-reviews and final approvals.
- ☐ Show the actual unittest result from latest main.
- ☐ Complete README.md, docs/design-decisions.md, and docs/ai-learning-log.md.
- ☐ Both fellows can explain their own code and what they learned using AI.
- ☐ Prepare a live demonstration including one invalid input, one valid lifecycle change, queue/report, and persistence.

Final individual reflection (each fellow answers)

1. Which feature did you personally implement, and how do you know it works?
2. What did AI teach you that you did not understand at the start?
3. Give one example where you verified, changed, or rejected an AI suggestion.
4. What did you learn from reviewing your partner's PR?
5. What would you improve next if the team had another hour?

11 | QUICK SELF-CHECK: ARE WE REALLY WORKING LIKE ENGINEERS?

- ☐ We understand the problem before writing code.
- ☐ We can explain each implementation decision.
- ☐ We use AI to learn, test, compare, and question—not blindly paste.
- ☐ We write tests for important behaviour and run them.
- ☐ We use Git branches with descriptive names and meaningful commits.
- ☐ We check each other's code before merging.
- ☐ We take responsibility for mistakes and fix them transparently.
- ☐ We communicate respectfully and share knowledge.

FINAL PRINCIPLE

“AI is your learning partner, not your replacement. Git is how you collaborate, not just where you upload code. A pull request invites careful review, not automatic approval. Your team succeeds when BOTH fellows understand what they have built and can demonstrate it with evidence.” _Ezennia Kingsley